Lecture L17

*Blind review — please score this lecture on row **L17** in the Excel form (1–10 per dimension).*

Lecture: Normalization Forms and Database Schema Design (1NF–BCNF)

The "Data Decay" Paradox

Imagine you are designing a system for a global logistics company. You create a table to track shipments, including the Driver's Name, their License Number, and the Truck ID. At first, it works. But as the company grows, you notice a strange phenomenon: when a driver retires and you delete their last shipment record, the system "forgets" that Truck #402 ever existed. Or, if a driver changes their phone number, you have to update 10,000 historical shipment rows, and if you miss one, your database now tells two different stories about the same person.

This is **Data Decay**. It isn't caused by hardware failure, but by **redundancy**. Normalization is the mathematical process of ensuring that every single fact is stored in exactly one place.

Learning Objectives

By the end of this lecture, you will be able to: 1. **Identify** and define Functional Dependencies (FDs) and Candidate Keys within a given relation. 2. **Detect** violations of 1NF, 2NF, 3NF, and BCNF in a database schema. 3. **Decompose** a non-normalized relation into a set of normalized relations to eliminate redundancy and anomalies. 4. **Evaluate** the trade-off between normalization (data integrity) and denormalization (query performance).

---

1. Prerequisites: The Tools of Normalization

To fix a schema, we need a way to map the "gravity" of the data—which attributes pull others along with them.

1.1 The Concept of Determination

Think of a Functional Dependency as a **lookup function**. If I give you a Student ID, you can provide exactly one Name. The ID "determines" the Name. However, if I give you a Name, you cannot reliably give me one Student ID (because of duplicates). This asymmetry is the engine of normalization.

1.2 Formal Definitions

- **Functional Dependency (FD):** We say $X \to Y$ (X functionally determines Y) if, for every pair of tuples t_1 and t_2 in a relation, whenever $t_1[X] = t_2[X]$, then $t_1[Y] = t_2[Y]$.
  - X is the **determinant** (the input).
  - Y is the **dependent** (the output).
- **Candidate Key:** A minimal set of attributes that uniquely identifies a tuple. It is the "smallest" possible determinant for the entire row.
- **Prime Attribute:** An attribute that is a member of at least one candidate key.
- **Non-prime Attribute:** An attribute that is not a member of any candidate key.

1.3 Worked Example: Identifying Dependencies

Consider the relation PROJECT_ASSIGNMENT(EmpID, ProjectID, EmpName, ProjectName).

The following functional dependencies exist: 1. EmpID → EmpName 2. ProjectID → ProjectName 3. {EmpID, ProjectID} → {EmpName, ProjectName}

Analysis: While EmpID can find the name, it cannot find the project. While ProjectID can find the project name, it cannot find the employee. Only the **combination** of both can pinpoint a specific assignment. * **Candidate Key:** {EmpID, ProjectID} * **Prime Attributes:** EmpID, ProjectID * **Non-prime Attributes:** EmpName, ProjectName

Try this yourself: You are designing a `BOOK` table with `ISBN, Title, AuthorID, AuthorName`. If `ISBN` determines everything, and `AuthorID` determines `AuthorName`, what are the candidate keys? What are the non-prime attributes?

---

2. First Normal Form (1NF): Atomicity

2.1 The Problem of "Packed" Data

Imagine a STUDENT table where the Courses column contains a comma-separated list: `"CS101, MATH202, PHYS101"`.

The Tension: It looks clean to the human eye, but it is a nightmare for the machine. To find students in "CS101", the database cannot use an index; it must perform a "full table scan," reading every single string and searching for a substring. If a student drops "MATH202", the system must pull the string, parse it, remove the element, and write the string back.

2.2 Formal Definition

A relation is in **First Normal Form (1NF)** if and only if the domain of every attribute contains only **atomic (indivisible) values**, and there are no repeating groups.

Analogy: 1NF is like a spreadsheet where every cell contains exactly one piece of data. If you have a "list" in a cell, you don't have a table; you have a table containing hidden tables.

2.3 Worked Example: Fixing 1NF

Violation: STUDENT(ID, Name, Courses) Row: `(101, "Alice", "CS101, MATH202")`

Normalization: Decompose the relation so each attribute holds a single value per row. STUDENT_COURSES(ID, N
Row 1: `(101, "Alice", "CS101")` Row 2: `(101, "Alice", "MATH202")`

Quick Check: If a table has a column Address containing Street, City, Zip, is it in 1NF? *(Answer: Technically yes, unless the business logic requires querying City independently, in which case it should be decomposed for utility.)*

---

3. Second Normal Form (2NF): Partial Dependencies

3.1 Intuition: The "Half-Key" Problem

Now that our data is atomic, we look at composite keys. 2NF prevents a non-prime attribute from being "half-tethered" to a key.

The Tension: If a table uses a composite key {A, B}, but attribute C only needs A to be identified, then C is repeated every time A appears with a different B. This is a **Partial Dependency**.

3.2 Formal Definition

A relation is in **Second Normal Form (2NF)** if: 1. It is in 1NF. 2. Every non-prime attribute is **fully functionally dependent** on every candidate key.

Analogy: Think of a 2NF violation as a "wrong address." If you have a table for `OrderItems(OrderID, ProductID, ProductDescription)`, the `ProductDescription` doesn't care about the `OrderID`; it only cares about the `ProductID`. Storing it here is like putting the manufacturer's address on every single receipt issued for that product.

3.3 Worked Example: Eliminating Partial Dependencies

Consider TEACH(Student, Course, Instructor, InstructorBirthday). * **Candidate Key:** {Student, Course} * **FDs:** {Student, Course} → Instructor AND Instructor → InstructorBirthday.

Analysis: InstructorBirthday is a non-prime attribute. It is determined by Instructor. Since Instructor is not the full candidate key, but is part of the relationship, we have a partial dependency.

Anomaly: If you delete the last student in a course, you accidentally delete the instructor's birthday from your system.

Normalization: 1. ENROLLMENT(Student, Course, Instructor) 2. INSTRUCTOR_DETAILS(Instructor, Instruc

Quick Check: If a relation has a single-column primary key and is in 1NF, is it automatically in 2NF? *(Answer: Yes, because you cannot have a partial dependency if the key cannot be split.)*

---

4. Third Normal Form (3NF): Transitive Dependencies

4.1 Intuition: The "Middleman"

Even in 2NF, we can have "hidden" dependencies. This happens when a non-prime attribute depends on another non-prime attribute.

The Tension: If Key → Attribute A → Attribute B, then Attribute B is transitively dependent on the key. This means B is only in the table because it's "friends" with A.

4.2 Formal Definition

A relation is in **Third Normal Form (3NF)** if it is in 2NF and no non-prime attribute is **transitively dependent** on a candidate key. Formally, for every non-trivial FD $X \to A$: 1. X is a superkey, OR 2. A is a prime attribute.

Analogy: 3NF is the "No Middlemen" rule. If you want to know the Department Name, you should ask the Department ID, not the Employee ID.

4.3 Worked Example: Eliminating Transitive Dependencies

Consider EMPLOYEE(EmpID, DeptID, DeptName). * **Candidate Key:** {EmpID} * **FDs:** EmpID → DeptID and DeptID → DeptName. * **Analysis:** DeptName is non-prime. It depends on DeptID (also non-prime). This is a transitive chain.

Anomaly: If you create a new department but haven't hired any employees yet, you cannot store the `DeptName` in this table because you have no `EmpID` to associate it with.

Normalization: 1. EMP(EmpID, DeptID) 2. DEPT(DeptID, DeptName)

Try this yourself: Look at a table `SESSIONS(SessionID, UserID, UserEmail)`. If `SessionID` → `UserID` and `UserID` → `UserEmail`, is this 3NF? If not, how do you split it?

---

5. Boyce-Codd Normal Form (BCNF)

5.1 Intuition: The Strictest Standard

BCNF is a stronger version of 3NF. It handles the rare but dangerous case where a table has **overlapping candidate keys**.

The Tension: 3NF allows a dependency $X \to A$ if A is a prime attribute (part of *some* key). BCNF says: "I don't care if A is prime; if X isn't a superkey, the dependency is forbidden."

5.2 Formal Definition

A relation is in **BCNF** if for every non-trivial functional dependency $X \to Y$, X must be a superkey.

The "Overlapping Key" Scenario: Imagine a clinic where: - A patient can see many doctors. - Each doctor specializes in only one area. - For any given area, a patient is assigned to only one doctor.

APPOINTMENT(Patient, Area, Doctor) * **Candidate Keys:** {Patient, Area} and {Patient, Doctor} * **FD:** Doctor → Area

The Problem: This is in 3NF because Area is a prime attribute (it's part of a candidate key). However, Doctor is **not** a superkey. This means we will redundantly store which doctor belongs to which area every time that doctor sees a new patient.

Normalization to BCNF: 1. DOCTOR_SPECIALTY(Doctor, Area) 2. PATIENT_DOCTOR(Patient, Doctor)

Summary: The Normalization Ladder

Form	Requirement	Eliminates…
1NF	Atomic values	Repeating groups / Lists
2NF	1NF + Full functional dependency	Partial dependencies (Half-key dependencies)
3NF	2NF + No transitive dependencies	Middleman dependencies (Non-prime → Non-prime)
BCNF	Every determinant must be a superkey	Anomalies from overlapping candidate keys

Final Engineering Trade-off: Normalization is a quest for **Integrity**. However, every time we decompose a table, we add a `JOIN` operation to our queries. In high-read environments (like a reporting dashboard), developers sometimes **denormalize** (intentionally move back to 3NF or 2NF) to trade storage space for raw speed. The goal is not "maximum normalization," but "correct normalization" for the specific workload.